International News Sentiment and Bias Analysis

Project Report

From scraped articles to a Spark and Delta Lake pipeline with a Power BI dashboard

DAV Semester Project
Group: 24L-2538 and 24L-2614
October 2026

Repository: github.com/rafayrizwanbabri/news-sentiment-and-analysis-of-bias

1. Introduction and objectives

News outlets often describe the same event in very different terms. This project measures those differences for international outlets across five topics: the Israel-Gaza conflict, the Russia-Ukraine war, US elections, Islamophobia and China. The aim is not to label an outlet as biased. It is to give readers measurable differences in sentiment, topic emphasis and keyword use that they can compare.

The repository covers the project from start to finish. It begins with an analysis stage in which articles were scraped, labelled for sentiment and keywords, and visualized. It continues with the Phase 1 proposal for a data engineering pipeline, the Phase 2 implementation of the Raw, Bronze and Silver layers on Delta Lake, and a Power BI dashboard. This report describes each part, the engineering decisions behind them, and what remains open. The questions guiding the work are:

- How does article volume vary by source and topic?
- How does sentiment vary by source and topic?
- How does coverage change over time?
- Which regions receive the most coverage?
- Which keywords dominate each sentiment category?

2. Repository overview

Path	Content
README.md	Description of the original analysis (sources, method, visualizations) and the Phase 2 pipeline
README_PIPELINE.md	Run instructions and scope of the pipeline
DAV_Phase1 (1).pdf	Phase 1 project proposal
pipeline/	PySpark modules: schemas, bronze_load, silver_transform, lakehouse (MERGE helper), audit, run_pipeline, and a pandas Gold script
Notebook/phase2_databricks_run.py	Databricks notebook that runs the pipeline with three parameters
raw/full_load, raw/incremental	Sample CSV files for the full and incremental loads (50 and 10 records)
news_full_load_*.json, news_incremental_*.json	Phase 1 raw samples in NewsAPI-style JSON (152 and 45 records)
Project.pbix	Power BI dashboard on the labelled article table
requirements.txt	Python dependencies

3. Data

Articles arrive as CSV files with twelve columns: Source, Link, Headline, Description, Timestamp, Date, Topic, Author, Region, Article_Content, Sentiment_Bias and Keywords. Sentiment labels and keywords come from the first stage of the project, which used fine-tuned transformer models and SHAP. Two ingestion patterns share the same code: a full load of historical data and incremental loads of new files. A backfill mode accepts a glob over historical folders.

File	Records	Description
raw/full_load/news_full_load.csv	50	Al Jazeera articles: Ukraine (32), Israel (11), US elections (7). Sentiment: 25 negative, 24 neutral, 1 positive
raw/incremental/news_incremental.csv	10	Ten articles delivered again with a changed source label and timestamp, which exercises the update path
news_full_load_2026-01-01_2026-09-15.json	152	Phase 1 NewsAPI-style baseline, six outlets, five topics
news_incremental_2026-09-16_2026-09-30.json	45	Phase 1 incremental sample

The CSV samples are deliberately small so the pipeline can be developed and tested cheaply on a free-tier workspace. The JSON files use a different layout from the CSV schema and are not read by the current pipeline.

4. Original analysis stage

The first stage of the project, described in the README, analysed more than 5,000 scraped articles from BBC, CNN, Al Jazeera, Fox and TRT on the five topics. The method had five steps:

- **Web scraping** with Selenium and BeautifulSoup to collect headlines, descriptions, timestamps, authors, regions and full article text.
- **Cleaning** of the scraped records so that fields were consistent.
- **Sentiment classification** with fine-tuned transformer models based on Twitter data, giving each article a Positive, Neutral or Negative label (field Sentiment_Bias).
- **Keyword extraction** with SHAP, which highlights the terms that contributed most to each sentiment prediction (field Keywords).
- **Visualization** with bar charts of sentiment by source, topic and region, pie charts of sentiment shares, line charts of sentiment over time, map charts of regional coverage, word clouds of keywords, and bivariate analysis.

The libraries used were Selenium, BeautifulSoup4, pandas, numpy, transformers, scikit-learn, SHAP, matplotlib, seaborn, wordcloud, plotly and nltk. The scraping and modelling scripts and the full 5,000-article dataset are not stored in this repository; the repository holds the labelled output in the CSV sample files and the Power BI dashboard built on it.

5. Project proposal (Phase 1)

The Phase 1 proposal turned the analysis into a pipeline design. It adds Dawn News to the outlets and proposes automated ingestion from news APIs, RSS feeds or GDELT. Following the Medallion architecture, it defines a Bronze layer for raw, traceable data, a Silver layer for cleaned, validated and deduplicated data, and a Gold layer of curated datasets for dashboards. Two ingestion patterns are specified: a full load that builds the historical baseline, and an incremental load that picks up articles published after the last successful run, removes duplicates with a deterministic article key, and updates existing records where required.

Load type	Estimated records	Estimated size
Full load	20,000 - 100,000	50 MB - 300 MB
Daily incremental	500 - 5,000	2 MB - 30 MB
Weekly incremental	3,000 - 20,000	10 MB - 100 MB

On sensitive data, the proposal notes that the dataset holds public editorial content and no identification numbers, financial data or credentials, but may contain names of authors, officials and public figures. It therefore treats article content and author information as potentially sensitive and plans to avoid exposing them in downstream datasets.

6. Architecture

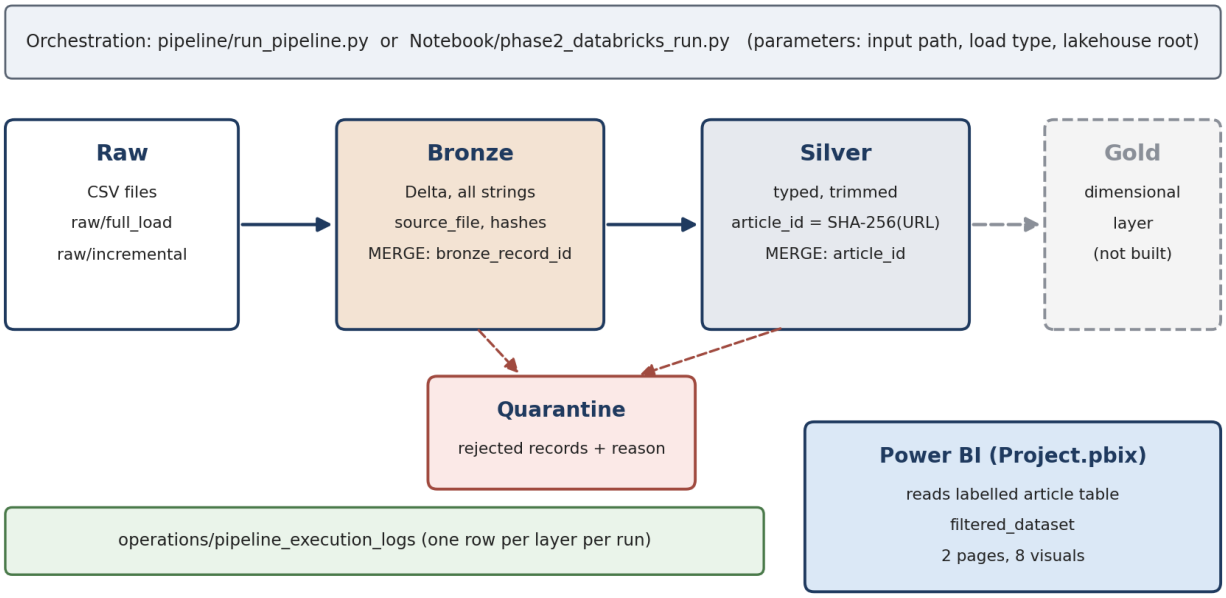


Figure 1. Data flow of the project. The Gold layer is shown dashed because it is not part of this submission.

Every layer is a Delta table under one configurable lakehouse root, so nothing in the code depends on a fixed path. One orchestrator (`pipeline/run_pipeline.py`, or the Databricks notebook `Notebook/phase2_databricks_run.py`) runs Bronze and then Silver with the input path, load type and lakehouse root passed as parameters.

Layer	Table	Write strategy	Purpose
Bronze	bronze/news_articles	MERGE on bronze_record_id	Source columns as strings, plus file name, load time, load type and hashes
Silver	silver/news_articles	MERGE on article_id	Typed, trimmed, one row per article
Quarantine	quarantine/news_articles	MERGE on record and reason	Rejected records with the reason
Audit	operations/pipeline_execution_logs	Append only	One row per layer per run

7. Bronze and Silver pipeline

Bronze. CSV files are read with an explicit schema declared in code and with every column as a string, so a change in the source shows up in Silver instead of being hidden at ingestion. Each record receives its source file, a load timestamp, the load type and two SHA-256 hashes: `bronze_record_id` (file path plus all columns) and `payload_hash` (business columns only). The header is inspected separately, and any unexpected column is recorded in `source_extra_columns`.

Silver. Text is trimmed, dates are parsed, and `article_id` is the SHA-256 hash of the article URL. `published_timestamp` is parsed from the Timestamp column, first as `d MMM yyyy`, then with the default parser, and finally from the Date column. Records with no URL, no headline, an unparseable date or unexpected source columns go to the quarantine table with a reason. If one batch contains the same URL more than once, a single row is kept.

Idempotent writes. Both layers use `MERGE INTO` keyed on a hash. A matched row is updated only when its `payload_hash` differs, so rerunning identical input changes nothing, and a changed article updates its row instead of creating a duplicate. The helper counts inserted and updated rows before the merge so the same numbers can be written to the audit log.

Audit log. Every run of each layer appends one row with the run id, layer, input path, start and end time, status, rows inserted, rows updated and a truncated error message when a run fails. Failures are logged before the exception is raised again, so a failed run still leaves a record.

8. Power BI dashboard

`Project.pbix` contains one imported table, `filtered_dataset`, holding the labelled articles, and two report pages with four visuals each. Every visual counts articles by the Source and Sentiment_Bias fields.

Page	Visual	Fields
Page 1 (by news channel)	Clustered column chart	Source on the axis, count of articles, split by Sentiment_Bias
	100 percent stacked column chart	Share of each sentiment within each source
	Ribbon chart	Rank of sentiment categories across sources
	Pie chart	Overall split of articles by sentiment
Page 2 (sentiments)	Clustered bar chart	Count of articles per sentiment category
	Clustered column chart	Sentiment on the axis, split by source
	100 percent stacked column chart	Share of each source within each sentiment
	Donut chart	Overall split of articles by sentiment

Together the pages show how much each outlet publishes and how its coverage splits between negative, neutral and positive tone, which answers the volume and sentiment questions for the source dimension. The visuals cover categorical breakdowns and shares; they do not yet include a time-series trend or a KPI card.

9. Decision log

Decision	Choice and reason
Partitioning	No table is partitioned. Partitioning pays off for very large tables, and these are far below the size where it helps; on tables this small it only creates many tiny files. If volume grows into the hundreds of millions of rows, a date-based partition or liquid clustering on the publication date is the first change, and OPTIMIZE with ZORDER on the common filter columns would be the first read optimization.
Schema drift	Bronze reads an explicit all-string schema and inspects the CSV header separately. Extra columns are kept in source_extra_columns, and Silver sends such records to quarantine with a reason instead of failing the batch or silently dropping data.
PII	Authors and article URLs are kept in Bronze and Silver for traceability only. The dashboard uses only the Source and Sentiment_Bias fields, so personal fields and article text are not displayed. A dedicated Gold layer that excludes these fields entirely is the planned next step.
All-string Bronze	Typing at ingestion would hide upstream changes. Bronze preserves exactly what was delivered, and conversion errors become visible quarantine reasons in Silver.
Deterministic keys	article_id is the hash of the URL and bronze_record_id is a hash of file and content, so reruns and backfills produce the same keys and the MERGE stays idempotent.
MERGE with payload hash	Overwriting would rewrite every file on each run. MERGE with a payload hash writes only new or changed rows and makes reruns harmless.
Quarantine instead of failure	A single bad record should not stop a batch. Rejected rows are stored with the reason, which also supports data-quality reporting.
Load modes	Full, incremental and backfill share one code path and differ only in input path and the stored load_type, which keeps behaviour consistent.

10. Spark execution analysis

The most complex transformation in the pipeline is the Bronze to Silver step. It reads the whole Bronze table, converts types, applies the validation rules, splits valid from invalid records, removes duplicate keys with a window function, and merges the result into Silver.

Narrow steps. Trimming, type conversion, date parsing, hash computation and the validation flags are row-level operations. Spark pipelines them inside one stage with no data movement. The scan reads only Bronze and the optimizer prunes columns that are not needed downstream.

Shuffles. The deduplication uses `row_number()` over a window partitioned by `article_id`. This forces an Exchange with hash partitioning on the key, a sort inside each partition and the window computation, which splits the job into a stage before and a stage after the shuffle. The Delta MERGE then joins the new rows with the existing table on the key. For small tables Spark can broadcast the new rows and avoid a second shuffle; for large tables it becomes a shuffle join, and only files containing matched rows are rewritten.

Pushdown. Filters on typed columns, such as the validity checks, can be pushed into the Delta scan, where file statistics let Spark skip files that cannot match.

11. Cost and FinOps evaluation

Measured run. The audit table records the start and end time of every layer, so the compute time of one incremental load can be read directly:

```
SELECT date(execution_start_time) AS run_date,
       SUM(unix_timestamp(execution_end_time) - unix_timestamp(execution_start_time)) AS seconds
FROM delta.`<lakehouse_root>/operations/pipeline_execution_logs`
GROUP BY 1 ORDER BY 1
```

Item	Value
Compute type used for development	REPLACE: e.g. Databricks Free Edition serverless, or Azure for Students job cluster
Typical daily incremental run (seconds, from the query above)	REPLACE with measured value
Credits or cost consumed by that run	REPLACE with measured value (Free Edition has no charge; state that if applicable)

Enterprise projection. The estimate below is illustrative and rests on stated assumptions. Published Azure Databricks list rates vary by tier and region; one public guide lists Jobs Compute at about $0.30 per DBU-hour and All-Purpose Compute at about $0.55, and the Azure pricing page should be checked for the current figure. Virtual machine charges are billed separately on classic compute.

Assumption	Value
Runs	Hourly, 24 x 365 = 8,760 runs per year
Billed time per run	5 minutes including cluster start (0.083 hours)
Cluster	Small job cluster, one driver and two workers, about 3 DBU per hour
DBU rate	$0.30 per DBU-hour (Jobs Compute)
VM cost	$1.00 per hour for the three nodes (assumed)
DBU cost per year	8,760 x 0.083 x 3 x $0.30 = about $660
VM cost per year	8,760 x 0.083 x $1.00 = about $730
Estimated total	About $1,400 per year, plus storage (negligible at this volume)

Cost scales linearly with billed minutes, so cluster start time is the hidden risk: a cluster that takes three minutes to start on every hourly run nearly doubles the bill for a short job. Optimizations, in order of impact:

- Skip empty runs: trigger the job on file arrival, or check for new files first, so no cluster starts when nothing changed.
- Process only new data: Silver currently reads the whole Bronze table, so a watermark on load_timestamp would make work grow with the new batch, not with the full history.
- Use Jobs Compute (or serverless jobs) instead of All-Purpose compute for scheduled runs, which is roughly half the DBU rate.
- Run OPTIMIZE nightly rather than on every hourly run, since compaction is only worth paying for after many small writes.
- Match the schedule to the business need: refreshing a daily-trend dashboard every four hours cuts compute cost by about three quarters.
- Use spot instances for workers and short auto-termination on any shared cluster.

12. Scope and limitations

The repository delivers the original analysis output, the Bronze and Silver layers, the audit and quarantine tables, and a Power BI dashboard on the labelled article table. The following parts of the proposal remain open:

- There is no Gold dimensional layer yet. The dashboard reads a single flat table, so it has no star schema, fact and dimension tables or table relationships, and it is not connected to Databricks Gold tables. The pandas script in the pipeline folder is not wired to the Spark layers.
- The dashboard does not yet include the audit and quarantine tables, a time-series trend, a regional view or a keyword view, so the time, region and keyword questions are not answered yet.
- Outlet names are not yet standardized in Silver; spellings such as BBC and BBC News, or the incremental sample label, would appear as separate sources.
- Many Al Jazeera timestamps carry a 'Last update' prefix that the parser rejects, so those records use the Date column instead.
- Silver reads the whole Bronze table on each run, and the JSON feed files are not yet ingested.
- The CSV samples contain one outlet and about 60 records, against the 20,000 to 100,000 estimated in the proposal, so cross-outlet comparison needs the larger dataset to be loaded through the same pipeline.
- The README refers to a Phase 2 data dictionary file that is not in the repository.

13. Conclusion

The project combines a completed analysis of news sentiment with a repeatable ingestion pipeline: a file entering the raw folder is ingested into Bronze, cleaned into Silver, with bad records isolated, every write safe to repeat and every run logged. A Power BI dashboard presents the sentiment breakdown by outlet. The next step is a Gold dimensional layer that feeds the dashboard directly and extends the logging, which would also allow the time, region and keyword questions to be answered from the pipeline.